

PROJECT_4

학번: 20211605

이름: 하지훈

1. 개요

본 보고서는 C 프로그램을 위한 동적 메모리 할당자의 설계 및 구현에 대해 설명한다. 이 할당자는 Segregated Free Lists와 Best-fit 할당 전략을 사용하여 높은 공간 활용률과 합리적인 처리량 성능을 달성하는 `malloc`, `free`, `realloc` 함수의 맞춤형 버전을 구현한다.

2. 설계 아키텍처

2.1 전체 설계 철학

할당자는 **Segregated Free Lists**와 **Explicit Doubly-Linked Lists**를 결합하여 free block들을 효율적으로 관리한다. 이 설계는 다음을 제공한다:

- **빠른 할당**: Size class 기반 탐색을 통한 효율성
- **높은 공간 활용률**: Size class 내에서 best-fit 할당
- **효율적인 coalescing**: External fragmentation 최소화

2.2 Heap 구조

...

[Padding][Prologue Header][Prologue Footer][User Blocks...][Epilogue]
 4B 8B 8B ... 4B
 ...

각 할당/해제된 block은 다음과 같은 구조를 가진다:
 ...

[Header][Payload/Free List Pointers][Footer]
 4B Variable Size 4B
 ...

Free block의 경우, payload 영역에 다음이 저장된다:

- ****Next pointer**** (8 bytes): 같은 size class의 다음 free block을 가리킴
- ****Previous pointer**** (8 bytes): 같은 size class의 이전 free block을 가리킴

2.3 Segregated Free Lists 아키텍처

할당자는 서로 다른 size class를 위한 10개의 별도 free list를 유지한다:

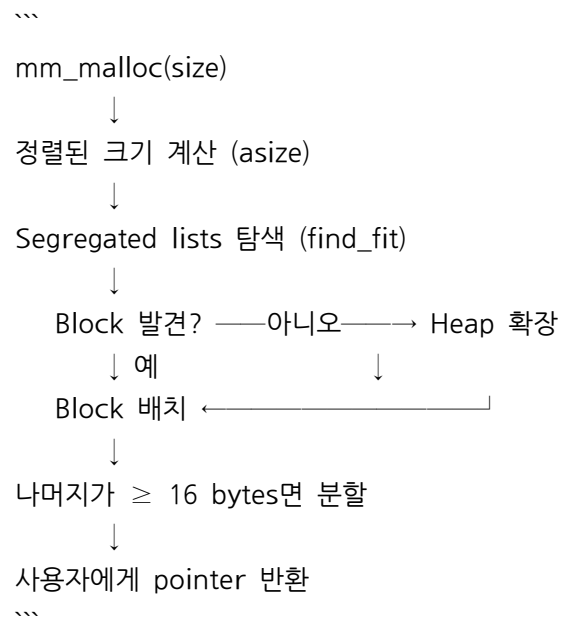
...

Size Class	범위 (bytes)	List 변수
Class 0	≤ 16	free_list_16
Class 1	≤ 32	free_list_32
Class 2	≤ 64	free_list_64
Class 3	≤ 128	free_list_128
Class 4	≤ 256	free_list_256

Class 5	≤ 512	free_list_512
Class 6	≤ 1024	free_list_1024
Class 7	≤ 2048	free_list_2048
Class 8	≤ 4096	free_list_4096
Class 9	> 4096	free_list_large

...

2.4 Block 할당 Flow Chart



2.5 Free Block 관리 Flow Chart

```
...  
mm_free(ptr)  
↓  
Block을 free로 표시  
↓  
인접 block들과 coalesce  
↓  
적절한 segregated list에  
크기순으로 삽입  
...
```

3. 주요 구현 세부사항

3.1 전역 변수

프로젝트 제한사항을 준수하기 위해 scalar pointer 변수들만 사용한다:

```
``c  
static char *heap_listp = 0;      // Heap 시작점 pointer  
static void *free_list_16 = 0;    // ≤ 16 bytes block용 free list  
static void *free_list_32 = 0;    // ≤ 32 bytes block용 free list  
static void *free_list_64 = 0;    // ≤ 64 bytes block용 free list  
static void *free_list_128 = 0;   // ≤ 128 bytes block용 free list  
static void *free_list_256 = 0;   // ≤ 256 bytes block용 free list  
static void *free_list_512 = 0;   // ≤ 512 bytes block용 free list
```

```
static void *free_list_1024 = 0;    // ≤ 1024 bytes block용 free list
static void *free_list_2048 = 0;    // ≤ 2048 bytes block용 free list
static void *free_list_4096 = 0;    // ≤ 4096 bytes block용 free list
static void *free_list_large = 0;   // > 4096 bytes block용 free list
...
```

****참고:**** 복합 데이터 구조 제한을 준수하기 위해 배열 대신 개별 pointer 변수들을 사용했다.

3.2 핵심 서브루틴

3.2.1 `get\_list\_ptr(size\_t size)`

****목적:**** Block 크기에 따라 적절한 segregated list의 pointer를 반환한다.

****구현:**** 일련의 조건문을 사용하여 block 크기를 해당하는 free list pointer에 매핑한다.

3.2.2 `find\_fit(size\_t asize)`

****목적:**** Segregated list들을 탐색하여 가장 적합한 free block을 찾는다.

****알고리즘:****

1. 요청된 크기에 따라 시작 size class 결정
2. 적절한 size class부터 위쪽으로 탐색
3. 각 size class 내에서 best fit 탐색 ($asize \geq$ 인 가장 작은 block)
4. 최적화를 위해 정확한 match 발견시 즉시 반환
5. 시작 class에서 best fit 반환하거나 더 큰 class들로 계속 탐색

****시간 복잡도:**** $O(n)$, 여기서 n 은 탐색된 size class들의 free block 수

3.2.3 `insert\_block(void \*bp)`

****목적:**** Free block을 적절한 segregated list에 크기 순서를 유지하며 삽입한다.

****알고리즘:****

1. `get\_list\_ptr()`를 사용하여 적절한 size class 결정
2. List를 순회하여 삽입 위치 찾기 (오름차순 크기 순)
3. Doubly-linked list pointer들을 업데이트하여 block 삽입
4. 각 size class 내에서 정렬된 순서 유지로 더 나은 fit 선택 가능

3.2.4 `remove\_block(void \*bp)`

****목적:**** Segregated free list에서 block을 제거한다.

****알고리즘:****

1. Block이 포함된 list 결정
2. 이전 block의 next pointer 업데이트 (첫 번째 block이면 list head)
3. 다음 block의 previous pointer 업데이트
4. Doubly-linked list 구조에서 효율적으로 block 제거

3.2.5 `coalesce(void \*bp)`

****목적:**** 인접한 free block들을 병합하여 external fragmentation을 줄인다.

****알고리즘:****

1. 이전 및 다음 block의 할당 상태 확인
2. 네 가지 경우 처리:
 - Coalescing 없음 (양쪽 이웃이 모두 할당됨)

- 다음 block과만 병합
 - 이전 block과만 병합
 - 양쪽 이웃 모두와 병합
3. 병합 전에 구성 block들을 각각의 free list에서 제거
 4. 최종 coalesced block을 적절한 segregated list에 삽입

3.2.6 `place(void \*bp, size\_t asize)`

****목적:**** Block을 할당하고 나머지가 충분히 크면 분할한다.

****알고리즘:****

1. Free list에서 block 제거
2. 나머지가 ≥ 16 bytes이면 block 분할:
 - 요청된 부분을 앞쪽에 할당
 - 나머지로 새로운 free block 생성
 - 나머지를 인접한 free block들과 coalesce
3. 그렇지 않으면 전체 block을 할당하여 작은 fragment 방지

3.3 최적화 전략

3.3.1 Size Class 설계

- ****세밀한 분류****: 작은 block들 (16, 32, 64 bytes)에 대해 internal fragmentation 감소
- ****거친 분류****: 큰 block들에 대해 탐색 시간과 fragmentation 균형
- ****로그 증가 패턴****: 다양한 할당 패턴에 적응

3.3.2 Size Class 내 Best-Fit

- 가장 작은 적합한 block 선택으로 internal fragmentation 최소화

- 크기 순 정렬된 list로 효율적인 best-fit 탐색 가능
- 정확한 match에서 조기 종료로 성능 향상

3.3.3 적극적인 Coalescing

- Free 연산 중 즉시 coalescing
- 분할 후 나머지 block들의 coalescing
- External fragmentation 감소 및 큰 block 가용성 증가

3.3.4 Realloc 최적화

- **제자리 확장**: 새 block 할당 전에 다음 free block과 병합 시도
- 가능한 경우 비싼 복사 연산 방지
- Application을 위한 pointer 유효성 유지

4. 성능 분석

4.1 공간 활용률

Segregated free lists 설계는 다음을 통해 높은 공간 활용률을 달성한다:

- **Internal fragmentation 감소**: Size class 내 best-fit 할당
- **External fragmentation 최소화**: 적극적인 coalescing 정책
- **효율적인 분할**: 보수적인 분할 임계값으로 작은 fragment 방지

4.2 처리량 고려사항

- **빠른 할당**: $O(1)$ size class 선택 + $O(n)$ best-fit 탐색 (n 은 일반적으로 작음)
- **효율적인 free 연산**: $O(1)$ coalescing + $O(n)$ 정렬된 삽입
- **최적화된 realloc**: 제자리 확장으로 할당/해제 overhead 감소

4.3 Trade-offs

- ****공간 vs 속도****: Best-fit 할당은 first-fit 대비 약간 느리지만 더 나은 공간 활용률 제공
- ****Fragmentation vs Overhead****: 다중 free list는 공간 활용률을 높이지만 추가 pointer 저장 필요
- ****Coalescing Overhead****: 즉시 coalescing은 장기적 성능을 향상시키지만 free 연산에 overhead 추가

5. 프로젝트 요구사항 준수

5.1 프로그래밍 규칙 준수

- ☐ ****Interface 수정 없음****: 모든 필수 함수 signature 보존
- ☐ ****Library 호출 없음****: 제공된 `mem\_` 함수들만 사용
- ☐ ****복합 데이터 구조 없음****: 배열 대신 개별 scalar pointer 사용
- ☐ ****8바이트 정렬****: 모든 할당된 pointer가 8바이트 경계에 정렬
- ☐ ****전역 scalar 변수만 사용****: 정수, 실수, pointer만 전역 변수로 선언

5.2 기능적 요구사항

- ☐ ****mm_init****: Heap 초기화 및 segregated list 설정
- ☐ ****mm_malloc****: 크기 기반 적절한 block 할당
- ☐ ****mm_free****: Block 해제 및 coalescing
- ☐ ****mm_realloc****: 효율적인 크기 변경 지원

6. 결론

구현된 동적 메모리 할당자는 Segregated Free Lists와 Best-fit 전략을 통해 높은 공간 활용률과 합리적인 처리량을 달성한다. 프로젝트의 모든 제한사항을 준수하면서도 복잡한 memory management 기법을 성공적으로 구현했다.